CSE 4309: Theory of Computing

Lecture 2 — Nondeterminism, Closure Properties, Regex to FA

Syed Rifat Raiyan
Lecturer
Department of Computer Science & Engineering
Islamic University of Technology, Dhaka, Bangladesh
Email: rifatraiyan@iut-dhaka.edu

Lecture Plan

The agenda for today

- What is a Nondeterministic Finite Automaton (NFA)?
- Converting NFAs to DFAs
- Regular Language Properties: Closure under $\circ$ and $*$
- Regular Expression (regex) to NFA conversion

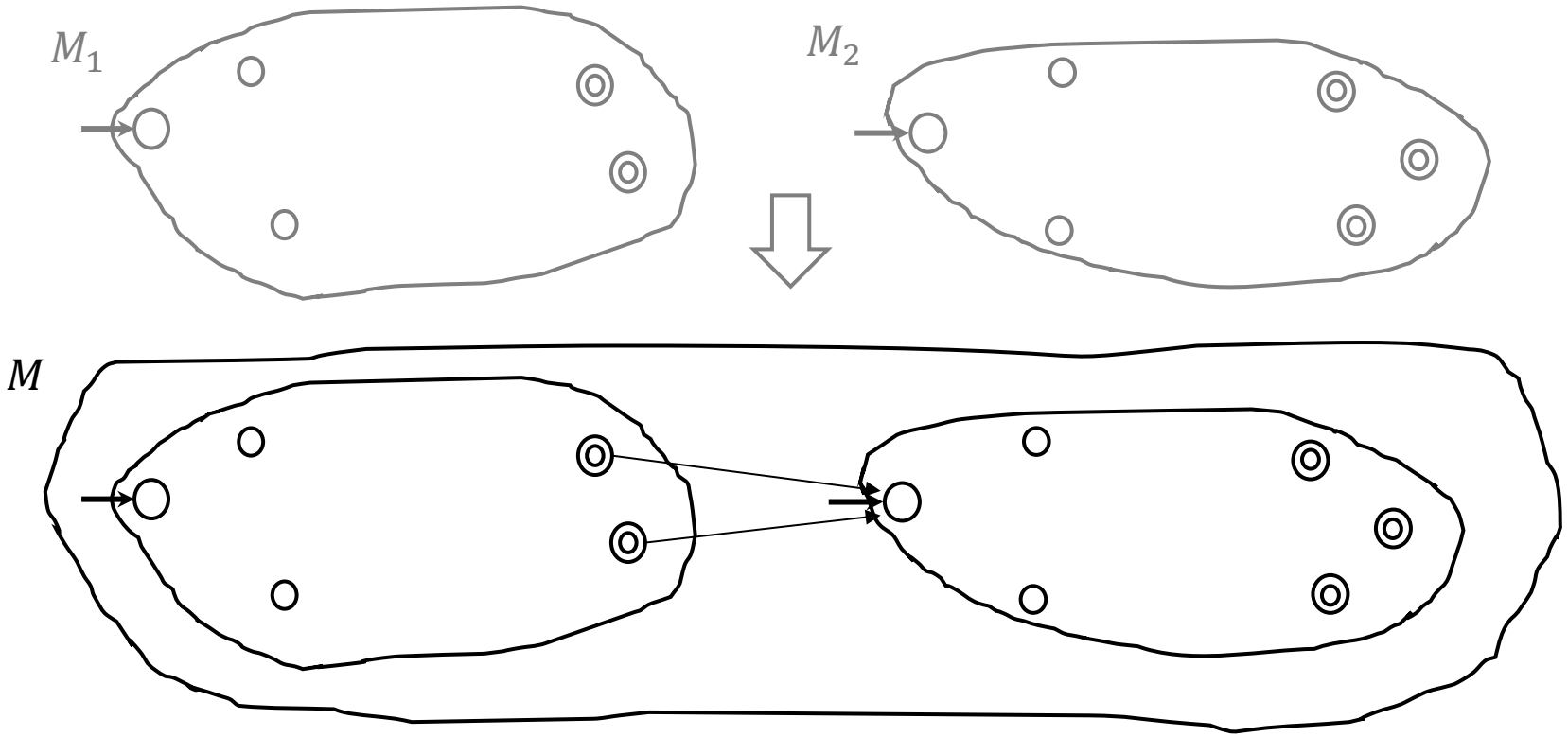
Regular Languages

Closure Properties

Theorem: **If A_1, A_2 are regular languages, then so is A_1A_2.** (closure under $\circ$)

Recall Proof Attempt: Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1
$M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2

Construct $M = (Q, \Sigma, \delta, q_0, F)$



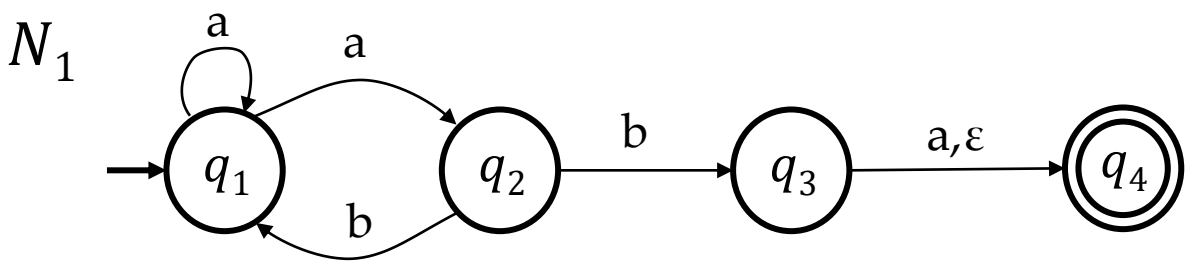
M should accept input w
if $w = xy$ where
M_1 accepts x and M_2 accepts y.

Doesn't work: Where to split w?

Hold off. Need **new** concept.

Nondeterministic Finite Automaton (NFA)

What is it?



New features of nondeterminism:

- multiple paths possible (0, 1 or many at each step)
- ε-transition is a "free" move without reading input
- Accept input if <u>some</u> path leads to ◎ accept

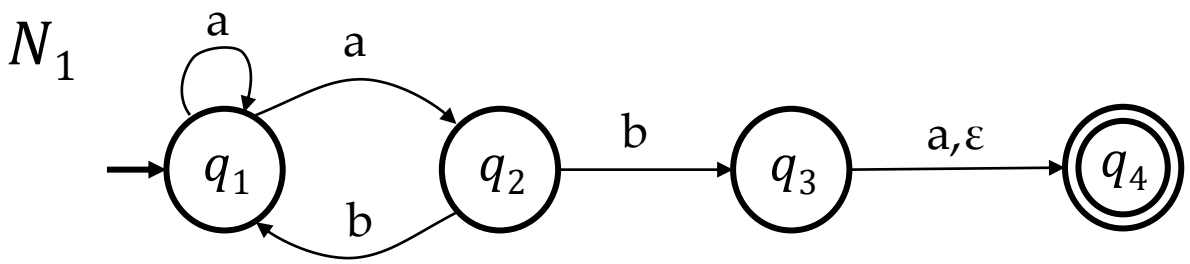
Example inputs:

- ab
- aa
- aba
- abb

Nondeterminism doesn't correspond to a physical machine we can build. However, it is useful mathematically.

Nondeterministic Finite Automaton (NFA)

Formal Definition



Defn: A nondeterministic finite automaton (NFA)

N is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$

states, alphabet, transition function, start state, accept states

- all same as before except δ
- $\delta: Q \times \underbrace{\Sigma}_{\Sigma \cup \{\varepsilon\}} \to \mathcal{P}(Q) = \{R | R \subseteq Q\}$ power set
- In the N_1 example: $\delta(q_1, \text{a}) = \{q_1, q_2\}$
  $\delta(q_1, \text{b}) = \emptyset$

Ways to think about nondeterminism:

<u>Computational:</u> Fork new parallel thread and accept if any thread leads to an accept state.

<u>Mathematical:</u> Tree with branches. Accept if any branch leads to an accept state.

<u>Magical:</u> Guess at each nondeterministic step which way to go. Machine ***always makes the right guess*** that leads to accepting, if possible.

Equivalence of NFA and DFA

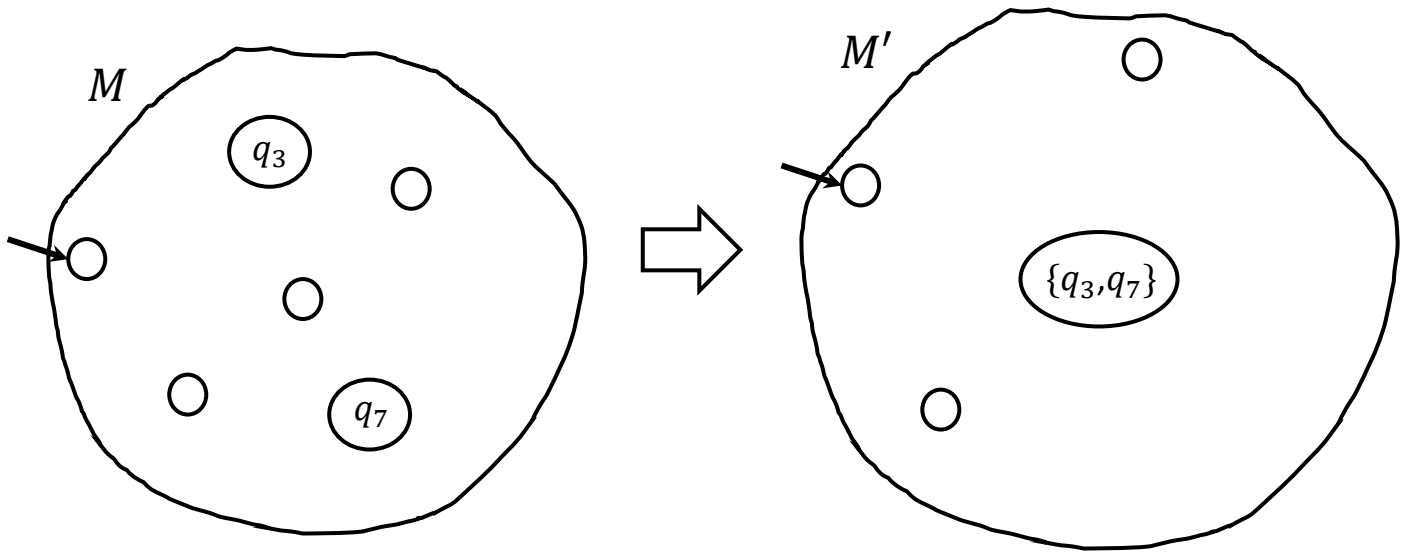
NFA ⇔ DFA

Theorem: If an NFA recognizes A, then A is regular.

Proof: Let NFA $M = (Q, \Sigma, \delta, q_0, F)$ recognize A
Construct DFA $M' = (Q', \Sigma, \delta', q_0', F')$ recognizing A

(Ignore the ε-transitions, can easily modify to handle them)

IDEA: DFA M' keeps track of the *subset of possible states* in NFA M.



Construction of M':

$Q' = \mathcal{P}(Q)$

$\delta'(\underbrace{R}_{R \in Q'}, a) = \{q \mid q \in \delta(r, a) \text{ for some } r \in R\}$

$q_0' = \{q_0\}$

$F' = \{R \in Q' \mid R \text{ intersects } F\}$

Mini Quiz

Number of states

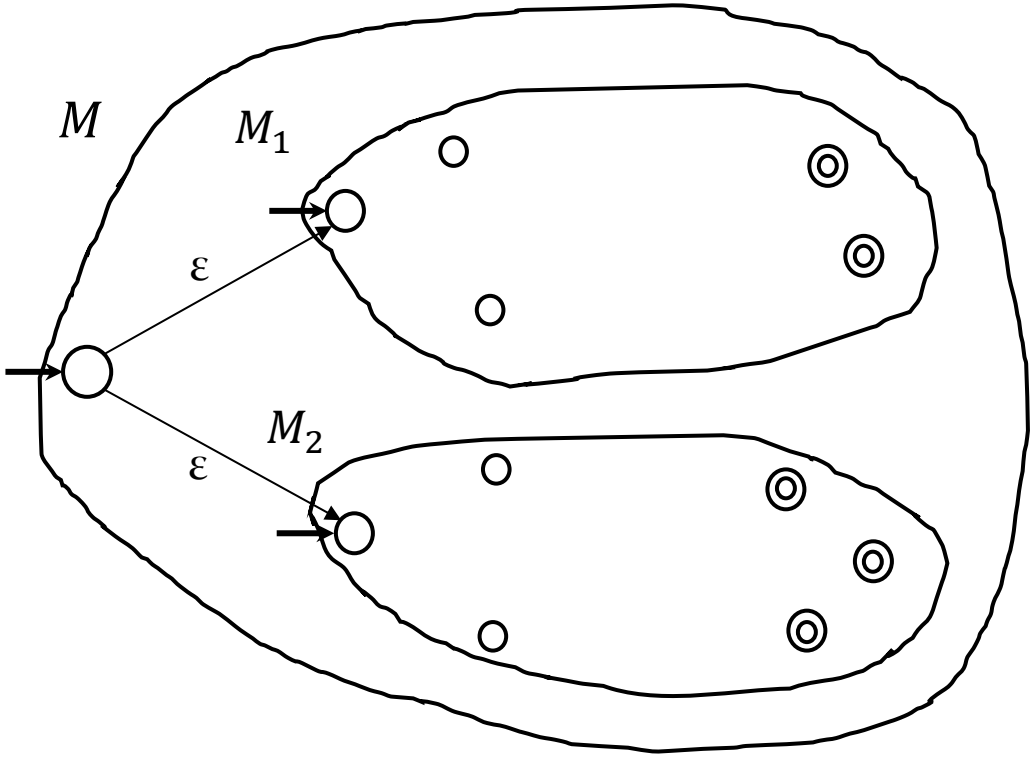
If the NFA M has n states, then how many states does the DFA M' have?

Closure Properties

Revisiting Closure under Union

Recall **Theorem:** If A_1, A_2 are regular languages, then $A_1 \cup A_2$ is also a regular language.
(The class of regular languages is closed under union)

New Proof (*sketch*): Given DFAs M_1 and M_2 recognizing A_1 and A_2
Construct NFA M recognizing $A_1 \cup A_2$



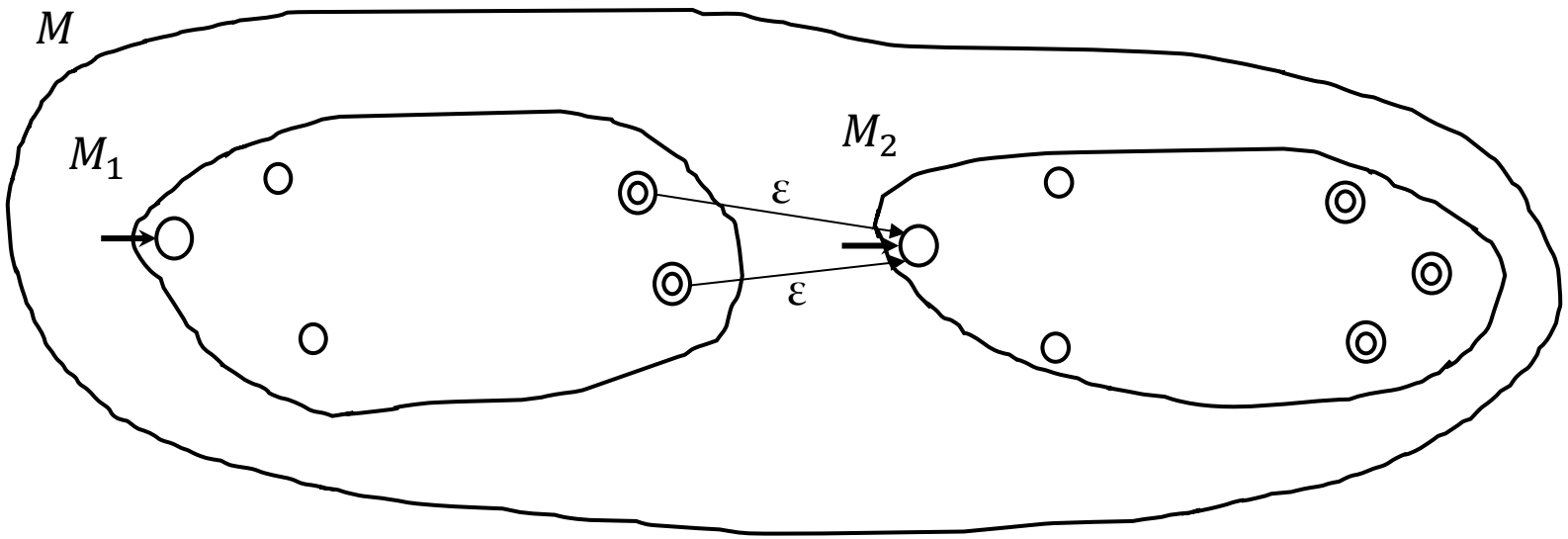
<u>Nondeterminism</u>
guessing instead
of parallelizing

Closure Properties

Closure under Concatenation

Theorem: If A_1, A_2 are regular languages, then $A_1 A_2$ is also a regular language.

Proof sketch: Given DFAs M_1 and M_1 recognizing A_1 and A_2
Construct NFA M recognizing $A_1 \circ A_2$



M should accept input w
if $w = xy$ where
M_1 accepts x and M_2 accepts y.

$w =$ x | y

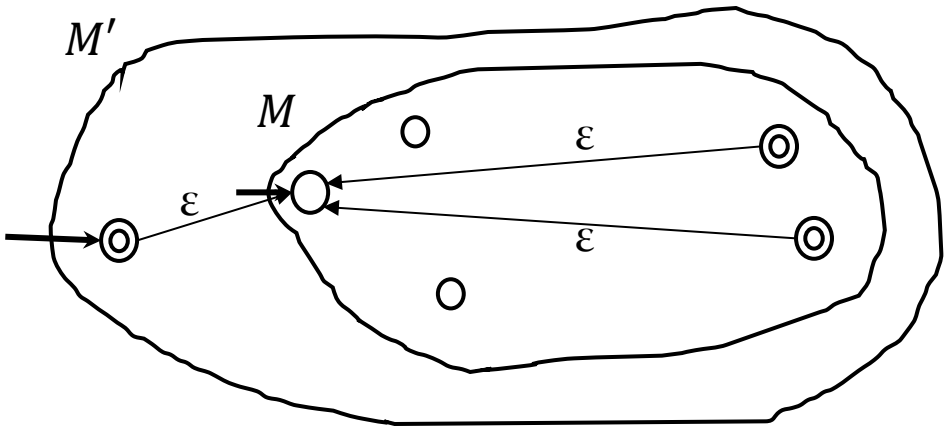
Nondeterministic M' has ***the option*** to jump to M_2 when M_1 accepts.

Closure Properties

Closure under Kleene Star

Theorem: If A is a regular language, then A^* is also a regular language.

Proof sketch: Given DFA M recognizing A
Construct NFA M' recognizing A^*



Make sure M' accepts ε

M' should accept input w
if $w = x_1 x_2 \ \dots \ x_k$
where $k \geq 0$ and M accepts each x_i

$w =$ x_1 x_2 x_3 x_4

Closure Properties

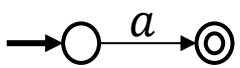
Closure under Kleene Star

Theorem: If R is a regular expr and $A = L(R)$ then A is regular

Proof: Convert R to equivalent NFA M:

If R is <u>atomic:</u> Equivalent M is:

$R = a$ for $a \in \Sigma$



$R = \varepsilon$

$R = \emptyset$

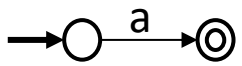
If R is <u>composite:</u>

$$\left.\begin{array}{l} R = R_1 \cup R_2 \\ R = R_1 \circ R_2 \\ R = R_1^* \end{array}\right\} \text{Use closure constructions}$$

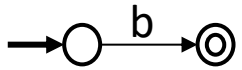
Example:

Convert $(a \cup ab)^*$ to equivalent NFA

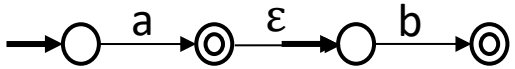
a:



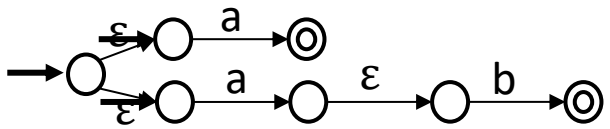
b:



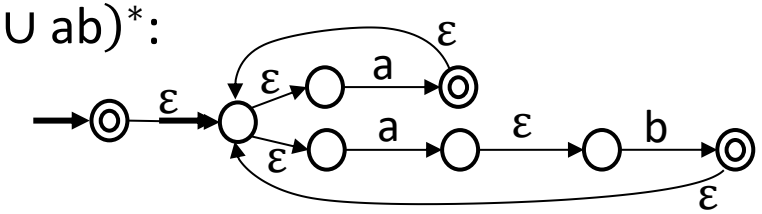
ab:



a ∪ ab:



$(a \cup ab)^*$:



Regular Expressions

Let's revisit the concept

Definition: Regular expression is a *compact* description of a set of strings.

Rules of precedence:

- Star binds *tighter*
  - ✓ $ab^* = a(b^*) \neq (ab)^*$
- Concatenation binds *tighter* than union
  - ✓ $a \circ b \cup c = (a \circ b) \cup c \neq a \circ (b \cup c)$

Regex to Language:

Let, $\Sigma = \{a, b, c, d\}$

$a \rightarrow L = \{a\}$

$abccb \rightarrow L = \{abccb\}$

$ab \cup cd \rightarrow L = \{ab, cd\}$

$a(b \cup c)d \rightarrow L = \{abd, acd\}$

$ab^*c \rightarrow L = \{ac, abc, abbc, abbbc, \ldots, abb \ldots bbc\}$

$a(b \cup \varepsilon)d \rightarrow L = \{abd, ad\}$

$\varepsilon \rightarrow L = \{\varepsilon\}$

$\varepsilon^* \rightarrow L = \{\varepsilon\}$

$\emptyset \rightarrow L = \{\ \}$

$\emptyset^* \rightarrow L = \{\varepsilon\}$

$$\varepsilon R = R\varepsilon = R$$

$$\emptyset R = R\emptyset = \emptyset$$

$$\emptyset \mid R = \emptyset \cup R = \{\ \} \cup R = R$$

Regular Expressions

Some more identities

$$R + R = R$$

$$R^* R^* = R^*$$

$$RR^* = R^* R$$

$$(R^*)^* = R^*$$

$$\epsilon + RR^* = \epsilon + R^* R = R^*$$

$$(PQ)^* P = P(QP)^*$$

$$(P + Q)^* = (P^* Q^*)^* = (P^* + Q^*)^*$$

$$(P + Q)R = PR + QR$$

$$R(P + Q) = RP + RQ$$

Regular Expressions

An example

Prove that

$$(1 + 00^*1) + (1 + 00^*1)(0 + 10^*1)(0 + 10^*1) = 0^*1(0 + 10^*1)^*$$

$$\begin{aligned}
(1 + 00^*1) + (1 + 00^*1)(0 + 10^*1)^*(0 + 10^*1) &= (1 + 00^*1)\big(\varepsilon + (0 + 10^*1)^*(0 + 10^*1)\big) \\
&= (1 + 00^*1)\big(\varepsilon + (0 + 10^*1)^+\big) \\
&= (1 + 00^*1)(0 + 10^*1)^* \\
&= (\varepsilon + 00^*)1(0 + 10^*1)^* \\
&= (\varepsilon + 0^+)1(0 + 10^*1)^* \\
&= 0^*1(0 + 10^*1)^* \\
&= 0^*1(0 + 10^*1)^* \\
&= \text{RHS}
\end{aligned}$$

Regular Expressions

Arden's theorem

Theorem: If P and Q are two regular expressions over Σ, and if P does **not** contain ε, then the following equation in R given by $R = Q + RP$, has a unique solution $R = QP^*$.

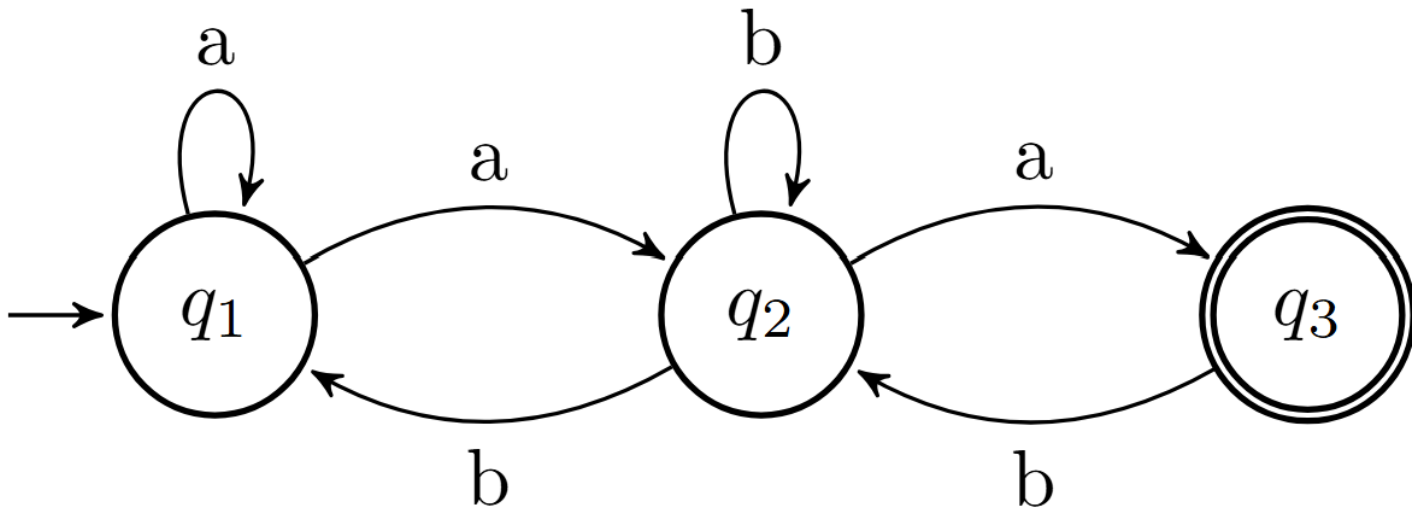
Proof:

$$
\begin{aligned}
R &= Q + RP \qquad (1) \\
&= Q + QP^*P \\
&= Q(\varepsilon + P^*P) \\
&= QP^*
\end{aligned}
$$

$$
\begin{aligned}
R &= Q + RP \\
&= Q + [Q + RP]P \\
&= Q + QP + RP^2 \\
&= Q + QP + [Q + RP]P^2 \\
&= Q + QP + QP^2 + RP^3 \\
&\vdots \\
&= Q + QP + QP^2 + \cdots + QP^n + RP^{n+1} \\
&= Q + QP + QP^2 + \cdots + QP^n + QP^*P^{n+1} \\
&= Q\left[\varepsilon + P + P^2 + \cdots + P^n + P^*P^{n+1}\right] \\
&= QP^*
\end{aligned}
$$

Regular Expressions

NFA to Regex example

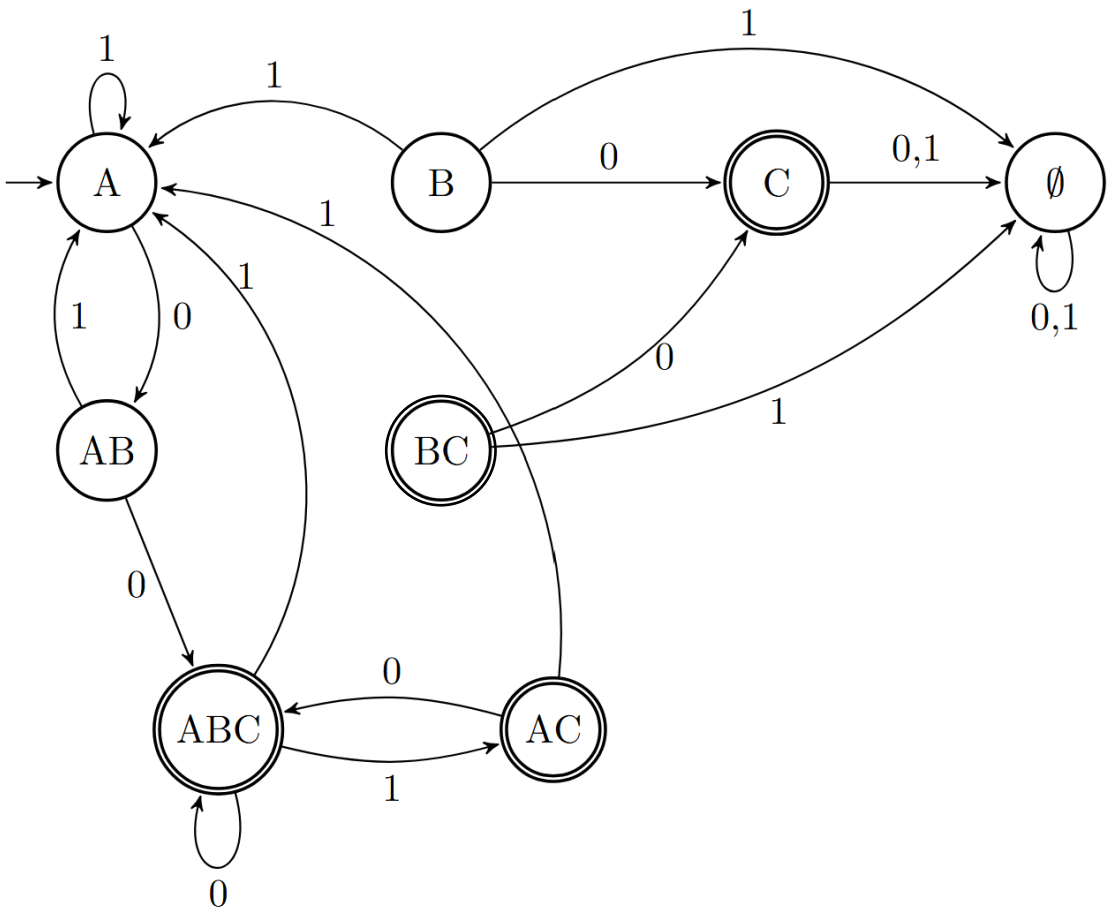
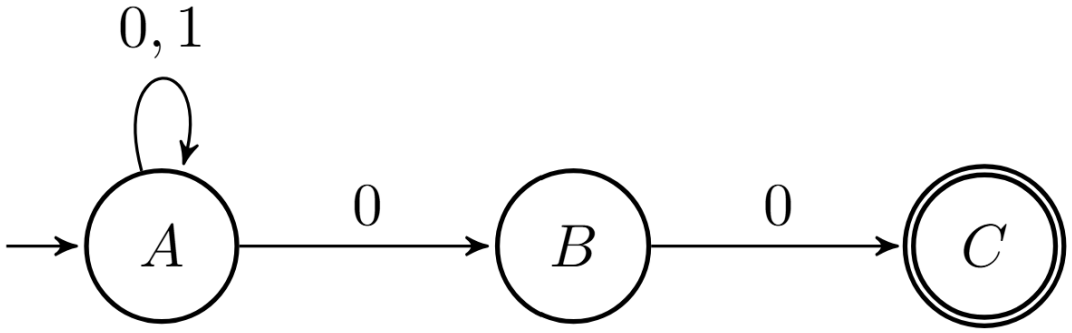
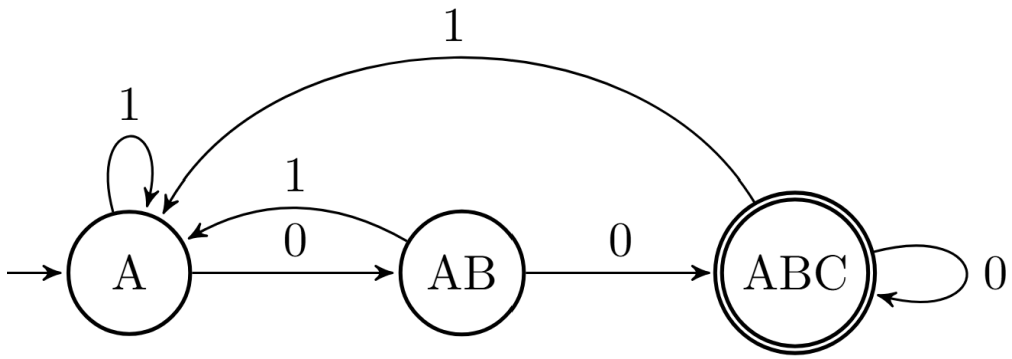


$$q_3 = (a + a(b + ab)^* b)^* a(b + ab)^* a$$

Regular Expressions

RL to Regex to NFA to DFA example

$$L = \{w \mid w \text{ ends with } 00\}$$



$$R = (0+1)^*00$$